

Zaawansowane Technologie Internetowe

Dokumentacja Projektu

System Rezerwacji Terminów Lekarskich

DocBooking

Autor: Artur Saganowski
Technologie: Spring Boot 3.4.3, Angular 20, PostgreSQL
Data: 30 czerwca 2026

Spis treści

1	Opis funkcjonalności aplikacji	2
1.1	Cel systemu	2
1.2	Funkcjonalności części serwerowej	2
1.3	Funkcjonalności klienta (Angular 20)	2
2	Architektura systemu	3
2.1	Diagram komponentów	3
2.2	Stos technologiczny	3
3	Diagramy UML	4
3.1	Diagram klas – model dziedziny	4
3.2	Diagram przypadków użycia	4
4	Opis kluczowych komponentów	5
4.1	Warstwa bezpieczeństwa – JWT	5
4.2	Programowanie aspektowe – Spring AOP	5
4.3	Komunikacja WebSocket – STOMP	6
5	REST API	7
6	Testy	8
6.1	Testy jednostkowe - backend (JUnit 5 + Mockito)	8
6.2	Testy integracyjne - kontroler REST	8
6.3	Testy frontendu - Angular (Jasmine)	8
7	Informacja uruchomieniowa	9
7.1	Wdrożenie produkcyjne	9
7.2	Wymagania – uruchomienie lokalne	9
7.3	Uruchomienie przez Docker Compose (lokalnie)	9
8	Podręcznik użytkownika	10
8.1	Rejestracja i logowanie	10
8.2	Lista lekarzy i rezerwacja wizyty	10
8.3	Historia wizyt i anulowanie	12
8.4	Panel lekarza	12
9	JavaDoc – przykłady	13

1 Opis funkcjonalności aplikacji

1.1 Cel systemu

DocBooking to aplikacja webowa klasy klient-serwer umożliwiająca pacjentom rezerwację wizyt lekarskich online. System zapewnia podgląd dostępnych terminów w czasie rzeczywistym dzięki technologii WebSocket, eliminując problem podwójnej rezerwacji tego samego slotu.

1.2 Funkcjonalności części serwerowej

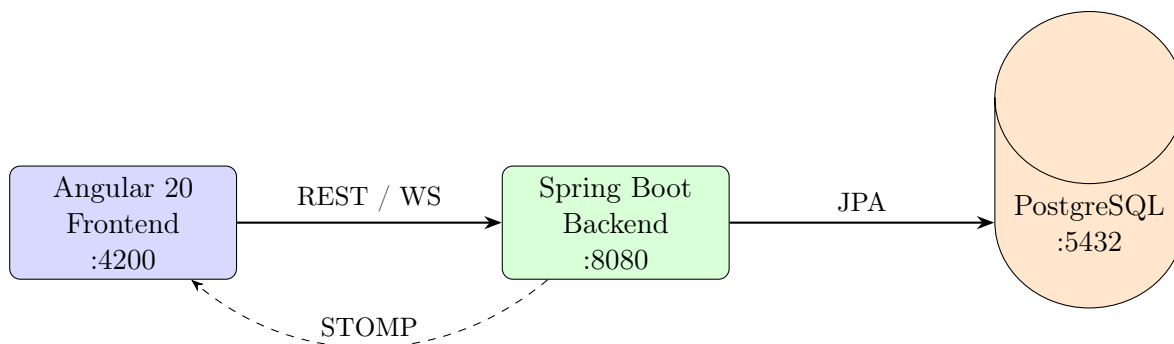
1. **Zarządzanie użytkownikami** – rejestracja pacjentów, logowanie, generowanie i walidacja tokenów JWT.
2. **Zarządzanie lekarzami** – pobieranie listy lekarzy, filtrowanie po specjalizacji.
3. **Zarządzanie slotami** – tworzenie slotów przez lekarza, pobieranie dostępnych terminów, zmiana statusu slotu (`AVAILABLE` / `BEING_RESERVED` / `BOOKED`).
4. **Rezerwacja wizyt** – tworzenie rezerwacji, anulowanie wizyty, historia wizyt pacjenta.
5. **WebSocket** – rozgłaszanie zmian statusu slotów do wszystkich klientów przeglądających terminy tego samego lekarza.
6. **Spring AOP** – automatyczne logowanie każdego wywołania serwisu (czas wykonania, użytkownik, błędy) oraz audit zdarzeń rezerwacji.
7. **Inicjalizacja danych** – seedowanie bazy przy pierwszym uruchomieniu: 3 lekarzy z terminami na 5 dni roboczych.

1.3 Funkcjonalności klienta (Angular 20)

1. **Rejestracja i logowanie** – formularze z walidacją, token JWT w `localStorage`.
2. **Lista lekarzy** – karty Angular Material z danymi lekarza.
3. **Widok terminów** – siatka slotów z live aktualizacją przez WebSocket.
4. **Rezerwacja** – dwuetapowy proces: oznaczenie slotu jako „rezerwowany”, następnie zapis wizyty.
5. **Historia wizyt** – lista wizyt z możliwością anulowania.
6. **Guard i Interceptor** – ochrona tras i automatyczne dołączanie tokenu JWT.

2 Architektura systemu

2.1 Diagram komponentów



Rysunek 1: Diagram komponentów systemu DocBooking

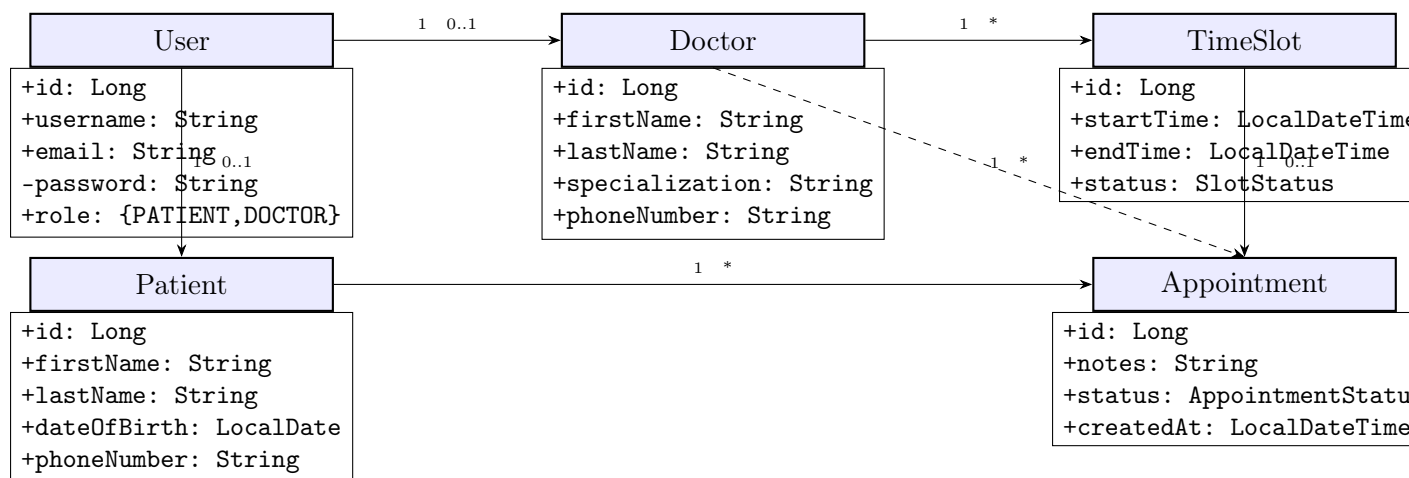
2.2 Stos technologiczny

Warstwa	Technologia	Rola
Frontend	Angular 20	SPA, routing, formularze
HTTP	HttpClient + Interceptor	Żądania REST z nagłówkiem JWT
WebSocket	STOMP.js + SockJS	Subskrypcja live aktualizacji
Backend	Spring Boot 3.4.3	REST API, WebSocket broker
Security	Spring Security + JWT	Autentykacja bezstanowa
ORM	Spring Data JPA	Mapowanie obiektowo-relacyjne
AOP	Spring AOP	Logowanie aspektowe
Baza	PostgreSQL 16	Relacyjna baza danych
Infra	Docker + Compose	Konteneryzacja wszystkich serwisów

Tabela 1: Stos technologiczny aplikacji

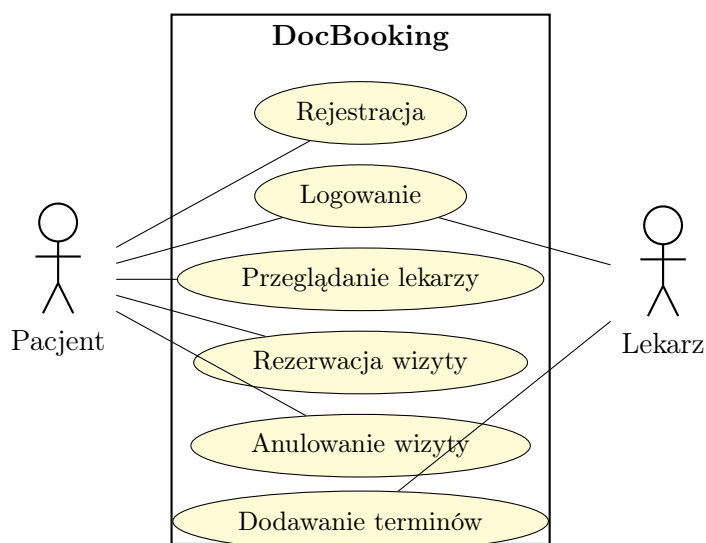
3 Diagramy UML

3.1 Diagram klas – model dziedziny



Rysunek 2: Diagram klas modelu dziedziny (backend)

3.2 Diagram przypadków użycia



Rysunek 3: Diagram przypadków użycia

4 Opis kluczowych komponentów

4.1 Warstwa bezpieczeństwa – JWT

Autentykacja opiera się na tokenach JWT podpisanych kluczem HMAC-SHA256. Po zalogowaniu klient przechowuje token w `localStorage` i dołącza go do każdego żądania przez interceptor.

Listing 1: Tworzenie tokenu JWT

```
1 private String createToken(Map<String, Object> claims, String subject) {
2     return Jwts.builder()
3         .claims(claims)
4         .subject(subject)
5         .issuedAt(new Date())
6         .expiration(new Date(System.currentTimeMillis() +
7             expiration))
8         .signWith(getSigningKey())
9         .compact();
10 }
```

4.2 Programowanie aspektowe – Spring AOP

Aspekt `LoggingAspect` automatycznie loguje każde wywołanie warstwy serwisowej bez modyfikacji kodu biznesowego.

Listing 2: Aspekt logowania – `LoggingAspect.java`

```
1 @Aspect
2 @Component
3 public class LoggingAspect {
4
5     @Pointcut("execution(* com.zti.doctorreservation.service.*(..))")
6     public void serviceLayer() {}
7
8     @Around("serviceLayer()")
9     public Object logServiceExecution(ProceedingJoinPoint joinPoint)
10         throws Throwable {
11         String method = joinPoint.getSignature().toShortString();
12         long start = System.currentTimeMillis();
13         Object result = joinPoint.proceed();
14         long elapsed = System.currentTimeMillis() - start;
15         log.info("[{}] {} completed in {}ms",
16             getCurrentUser(), method, elapsed);
17         return result;
18     }
19
20     @After("execution(* com.zti.doctorreservation.service"
21         + ".AppointmentService.book(..))")
22     public void logAppointmentBooked(JoinPoint joinPoint) {
23         log.info("[AUDIT] Appointment booked by: {}", getCurrentUser());
24     }
25 }
```

4.3 Komunikacja WebSocket – STOMP

Listing 3: Rozgłaszanie aktualizacji slotu – TimeSlotService.java

```

1 private void broadcastSlotUpdate(Long doctorId, TimeSlot slot) {
2     messagingTemplate.convertAndSend(
3         "/topic/slots/" + doctorId, slot);
4 }

```

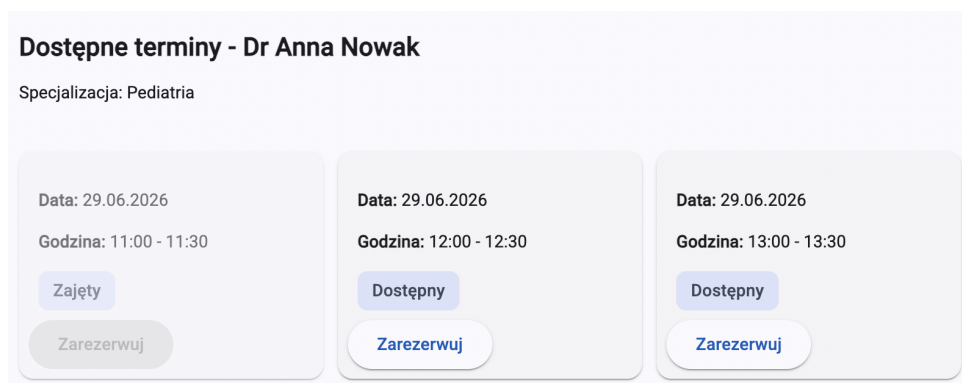
Listing 4: Subskrypcja WebSocket – websocket.ts (Angular)

```

1 subscribeToSlots(doctorId: number): Observable<any> {
2     return new Observable(observer => {
3         let stompSub: StompSubscription | undefined;
4         let timeoutId: ReturnType<typeof setTimeout> | undefined;
5
6         const checkAndSubscribe = () => {
7             if (this.client?.connected) {
8                 stompSub = this.client.subscribe(`/topic/slots/${doctorId}`, (
9                     msg: IMessage) => {
10                     observer.next(JSON.parse(msg.body));
11                 });
12             } else {
13                 timeoutId = setTimeout(checkAndSubscribe, 500);
14             }
15         };
16
17         checkAndSubscribe();
18
19         return () => {
20             clearTimeout(timeoutId);
21             stompSub?.unsubscribe();
22         };
23     });
24 }

```

Kluczową cechą systemu jest eliminacja problemu podwójnej rezerwacji tego samego terminu. Gdy pacjent A przegląda dostępne sloty lekarza i w tym samym czasie pacjent B rezerwuje jeden z nich, backend natychmiast rozsyła aktualizację przez WebSocket do wszystkich klientów subskrybujących temat. Pacjent A bez odświeżania strony widzi, że dany termin zmienił status na **Zajęty**.



Rysunek 4: Widok terminów po aktualizacji przez WebSocket – slot zmienił status na Zajęty

5 REST API

Metoda	Ścieżka	Rola
POST	/api/auth/register	Publiczny
POST	/api/auth/login	Publiczny
GET	/api/doctors	AUTH
GET	/api/doctors/me	DOCTOR
GET	/api/doctors/{id}	AUTH
GET	/api/doctors/specialization/{spec}	AUTH
GET	/api/doctors/{id}/slots	AUTH
GET	/api/doctors/{id}/slots/all	DOCTOR
POST	/api/doctors/{id}/slots	DOCTOR
POST	/api/appointments	PATIENT
GET	/api/appointments/my	PATIENT
GET	/api/appointments/doctor/{doctorId}	DOCTOR
PUT	/api/appointments/{id}/cancel	PATIENT/DOCTOR
POST	/api/appointments/slots/{id}/reserve	PATIENT

Tabela 2: Endpointy REST API

```

/**
 * Zwraca profil aktualnie zalogowanego lekarza.
 *
 * @param auth dane uwierzytelnienia z kontekstu Spring Security
 * @return profil lekarza lub 404 gdy lekarz nie istnieje
 */
@GetMapping("/me")
@PreAuthorize("hasRole('DOCTOR')")
public ResponseEntity<Doctor> getMyProfile(Authentication auth) {
    return doctorRepository.findAll().stream()
        .filter(d -> d.getUser() != null &&
            d.getUser().getUsername().equals(auth.getName()))
        .findFirst()
        .map(ResponseEntity::ok)
        .orElse(ResponseEntity.notFound().build());
}

```

Rysunek 5: Przykładowy endpoint REST API

6 Testy

6.1 Testy jednostkowe - backend (JUnit 5 + Mockito)

Testy jednostkowe weryfikują logikę serwisów w izolacji od bazy danych przy użyciu atrap (Mockito). Przykładowe przypadki testowe dla AuthService:

- `register_ShouldCreatePatientAndReturnToken` – poprawna rejestracja zwraca token JWT i tworzy profil pacjenta,
- `register_ShouldThrow_WhenUsernameExists` – duplikat nazwy użytkownika zgłasza `IllegalArgumentException`.

6.2 Testy integracyjne - kontroler REST

Test integracyjny uruchamia pełny kontekst Spring Boot (`@SpringBootTest`) i weryfikuje endpoint `POST /api/auth/register` przez `MockMvc`. Test sprawdza kod odpowiedzi 200 oraz obecność tokenu JWT w odpowiedzi JSON.

6.3 Testy frontendu - Angular (Jasmine)

Testy serwisów Angular weryfikują logikę HTTP i zarządzanie stanem bez uruchamiania przeglądarki.

```
it('logout() powinien usunac token i ustawic isLoggedIn na false', () => {
  localStorage.setItem('token', 'fake.jwt.token');
  localStorage.setItem('role', 'PATIENT');
  localStorage.setItem('username', 'testuser');

  service.logout();

  expect(localStorage.getItem('token')).toBeNull();
  expect(service.isLoggedIn()).toBeFalse();
  expect(service.currentRole()).toBeNull();
});

it('register() powinien zapisac token po rejestracji', () => {
  const registerData = {
    username: 'newuser',
    email: 'new@example.com',
    password: 'haslo123',
    firstName: 'Jan',
    lastName: 'Kowalski',
  };

  service.register(registerData).subscribe();

  const req = httpMock.expectOne(r => r.url.includes('/auth/register'));
  expect(req.request.method).toBe('POST');
  req.flush({ token: 'new.jwt.token', username: 'newuser', role: 'PATIENT' });

  expect(localStorage.getItem('token')).toBe('new.jwt.token');
  expect(service.isLoggedIn()).toBeTrue();
});
```

Rysunek 6: Przykładowe testy jednostkowe w Angular

7 Informacja uruchomieniowa

7.1 Wdrożenie produkcyjne

Aplikacja jest wdrożona i dostępna pod adresem:

<https://perpetual-surprise-production-b801.up.railway.app>

Wdrożenie realizowane jest na platformie Railway z wykorzystaniem kontenerów Docker.

7.2 Wymagania – uruchomienie lokalne

- Docker Engine $\geq$ 24.0 i Docker Compose $\geq$ 2.0
- Wolne porty: 4200 (frontend), 8080 (backend), 5432 (PostgreSQL)

7.3 Uruchomienie przez Docker Compose (lokalnie)

W katalogu projektu wykonaj:

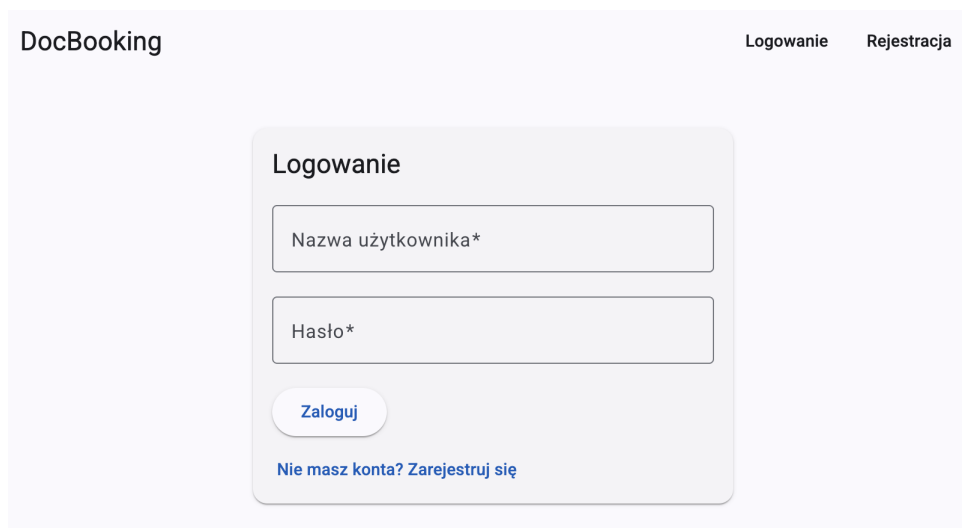
```
docker compose up --build
```

Aplikacja będzie dostępna pod adresem <http://localhost:4200>.

8 Podręcznik użytkownika

8.1 Rejestracja i logowanie

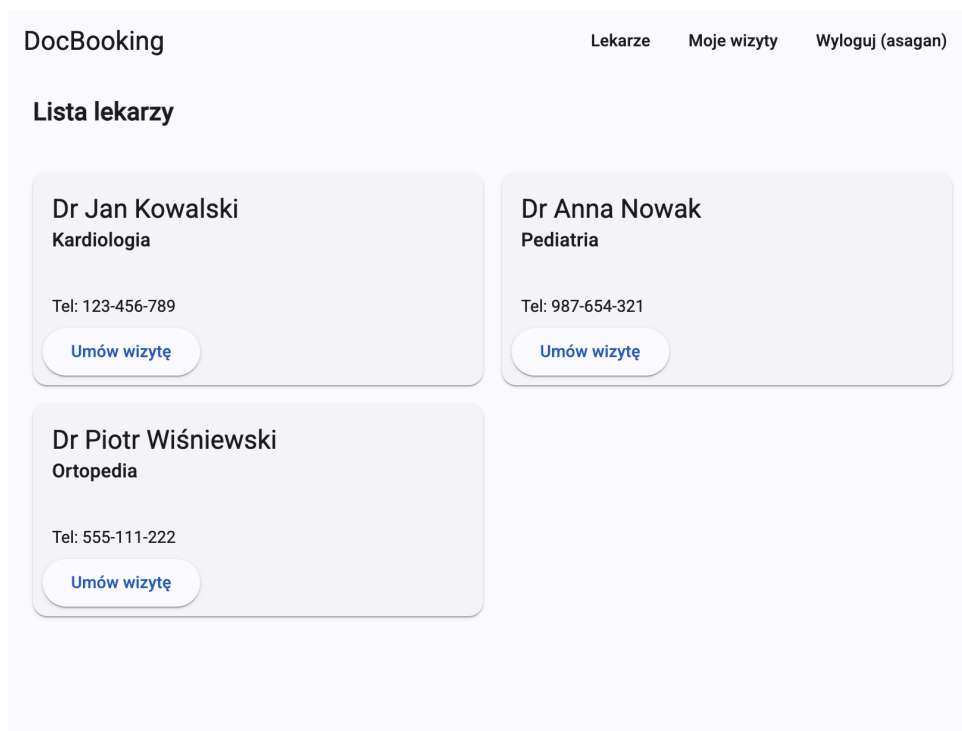
1. Otwórz aplikację, kliknij **Rejestracja**, wypełnij formularz i zatwierdź.
2. Logowanie odbywa się przez **Logowanie** – po poprawnym uwierzytelnieniu system przekierowuje do widoku odpowiedniego dla roli.



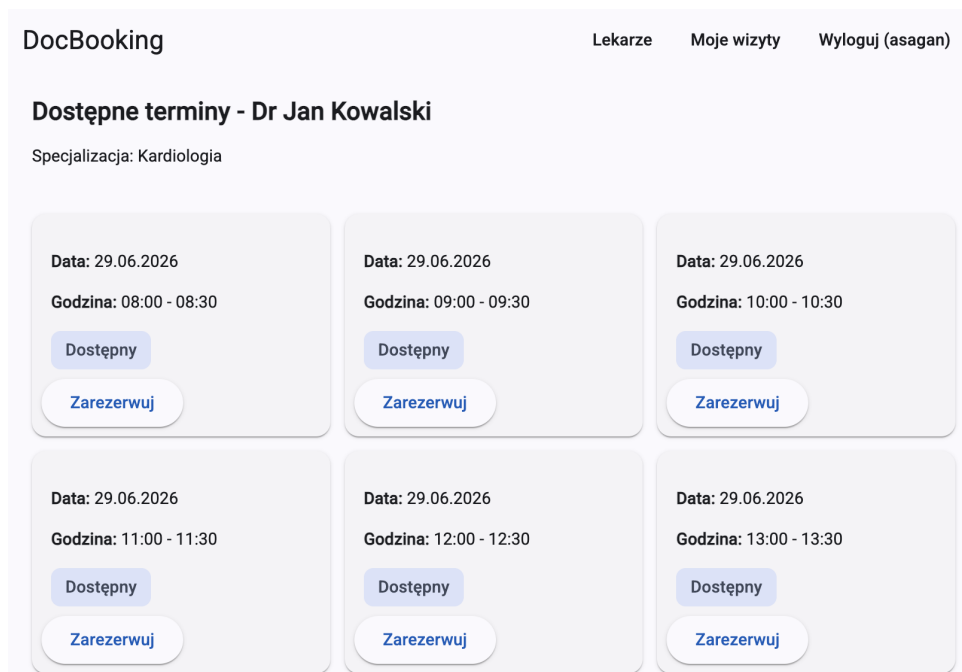
Rysunek 7: Ekran logowania

8.2 Lista lekarzy i rezerwacja wizyty

1. Przejdź do **Lekarze** i wybierz lekarza klikając **Umów wizytę**.
2. Na widoku terminów sloty aktualizują się w czasie rzeczywistym przez WebSocket.
3. Kliknij **Zarezerwuj** – wizyta zostaje zapisana.



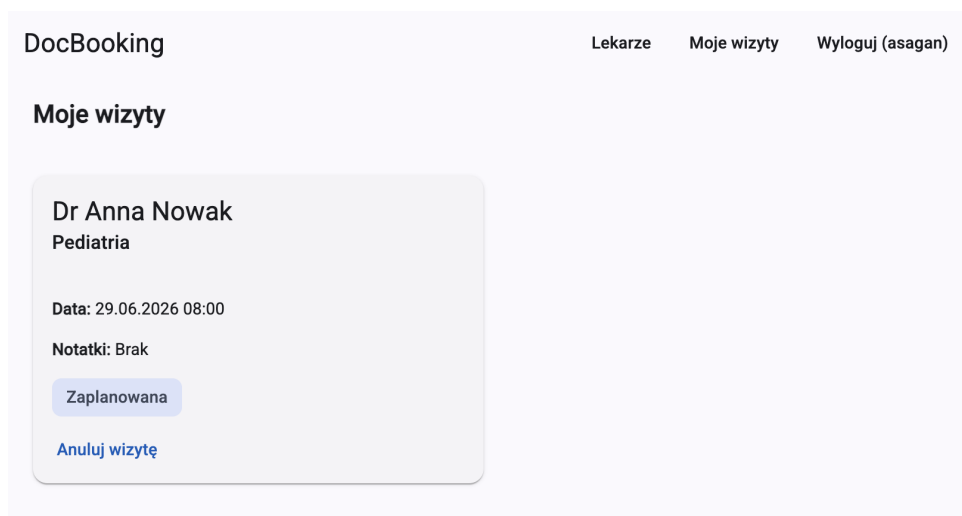
Rysunek 8: Lista lekarzy



Rysunek 9: Widok dostępnych terminów

8.3 Historia wizyt i anulowanie

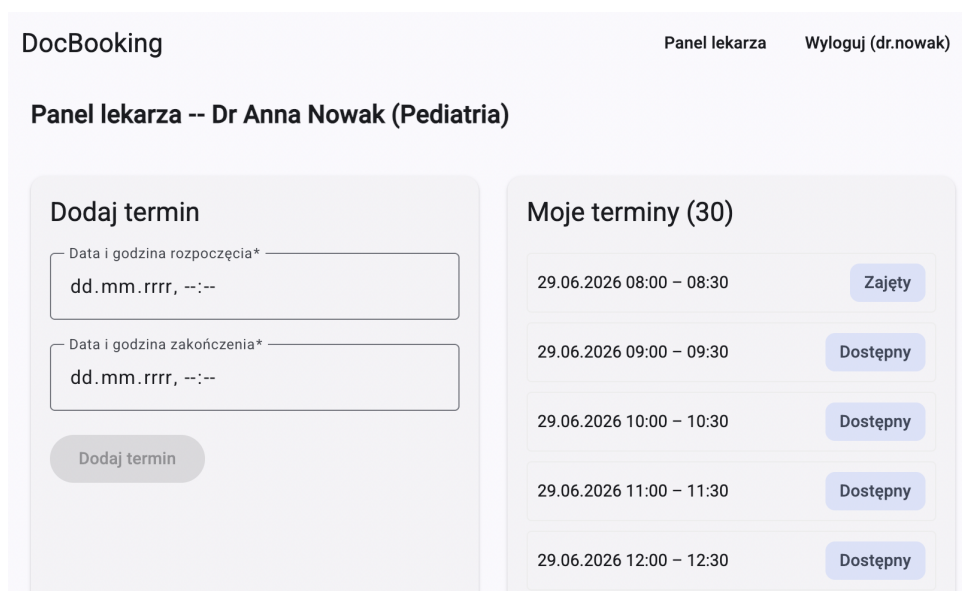
Pacjent przechodzi do **Moje wizyty** i może anulować zaplanowaną wizytę – slot wraca do puli dostępnych.



Rysunek 10: Historia wizyt pacjenta

8.4 Panel lekarza

Lekarz po zalogowaniu widzi zakładkę **Panel lekarza**, gdzie może dodawać nowe terminy wizyt.



Rysunek 11: Panel lekarza – zarządzanie terminami

9 JavaDoc – przykłady

Listing 5: JavaDoc – AppointmentService.java

```

1  /**
2   * Serwis obsługuje logikę biznesową rezerwacji wizyt.
3   * Po każdej zmianie statusu slotu rozpoczyna aktualizację
4   * przez WebSocket do wszystkich podłączonych klientów.
5   */
6  @Service
7  public class AppointmentService {
8
9      /**
10       * Rezerwuje wizytę dla zalogowanego pacjenta.
11       *
12       * @param request obiekt zawierający ID slotu i notatki
13       * @return zapisana wizyta
14       * @throws IllegalStateException gdy slot jest niedostępny
15       * @throws IllegalArgumentException gdy slot nie istnieje
16       */
17     public Appointment book(AppointmentRequest request) { ... }
18
19     /**
20      * Anuluje wizytę i zwalnia slot czasowy.
21      *
22      * @param appointmentId ID wizyty do anulowania
23      * @return wizyta ze statusem CANCELLED
24      */
25     public Appointment cancel(Long appointmentId) { ... }
26 }

```

Wszystkie klasy i metody warstwy serwisowej, kontrolerów oraz komponentów bezpieczeństwa zostały opisane komentarzami JavaDoc. Dokumentacja zawiera opisy parametrów (@param), wartości zwracanych (@return) oraz zgłaszanych wyjątków (@throws). Wygenerowana dokumentacja HTML jest dostępna lokalnie po uruchomieniu `mvn javadoc:javadoc`.

doctor-reservation 0.0.1-SNAPSHOT API

Packages	
Package	Description
com.zti.doctorreservation	
com.zti.doctorreservation.aspect	
com.zti.doctorreservation.config	
com.zti.doctorreservation.controller	
com.zti.doctorreservation.dto	
com.zti.doctorreservation.model	
com.zti.doctorreservation.repository	
com.zti.doctorreservation.security	
com.zti.doctorreservation.service	

Copyright © 2026. All rights reserved.

Rysunek 12: Wygenerowana dokumentacja JavaDoc